

Kubernetes-03

Tasks:

1. Create a ClusterIP service for an Apache web server pod.
2. Expose an Nginx pod externally using a NodePort service.
3. Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.
4. Scale the ReplicationController from 3 replicas to 5 replicas.
5. Create a ReplicaSet to manage pods based on multiple labels (prod and test).
6. Deploy a ReplicaSet that excludes pods with the label backend.
7. Test load balancing across multiple pods using a NodePort service.
8. Delete a ReplicationController without affecting the running pods.

Task :1 Create a ClusterIP service for an Apache web server pod.

Step 1 — Create Apache Pod

Create file:

```
vi apache-pod.yaml
```

```
ubuntu@master:~$ vi apache-pod.yaml
[ubuntu@master:~$ cat apache-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: apache-pod
  labels:
    app: apache
spec:
  containers:
  - name: apache-container
    image: httpd
    ports:
    - containerPort: 80

ubuntu@master:~$
```

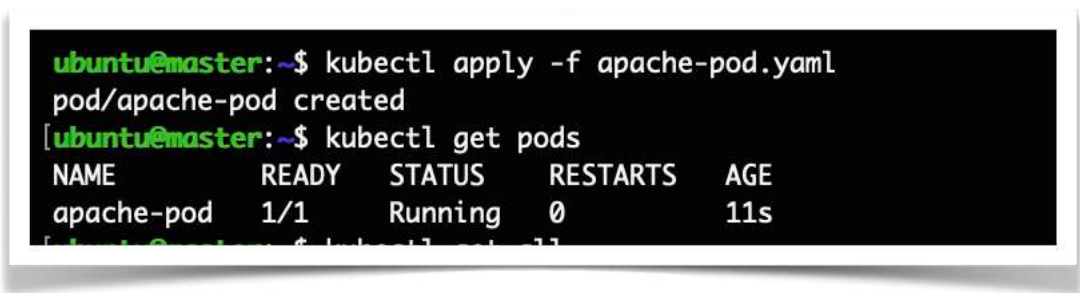
Apply:

```
kubectl apply -f apache-pod.yaml
```

```
kubectl get pods
```

Wait until:

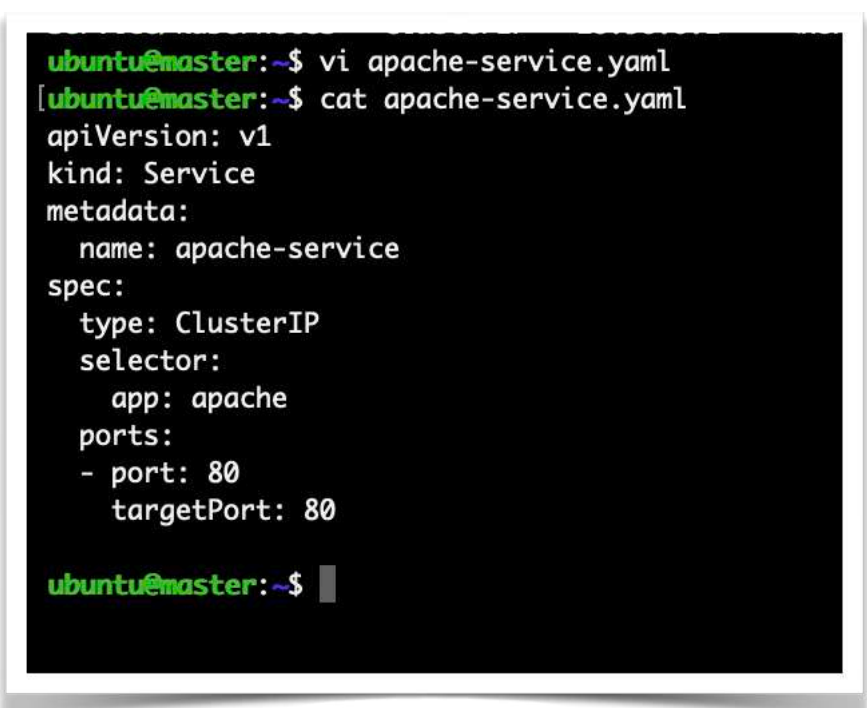
```
apache-pod    Running
```

A terminal window with a black background and green text. The first command is 'kubectl apply -f apache-pod.yaml' which outputs 'pod/apache-pod created'. The second command is 'kubectl get pods' which outputs a table with columns: NAME, READY, STATUS, RESTARTS, and AGE. The table shows one pod named 'apache-pod' with a READY status of '1/1', a STATUS of 'Running', 0 restarts, and an age of '11s'.

```
ubuntu@master:~$ kubectl apply -f apache-pod.yaml
pod/apache-pod created
[ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
apache-pod    1/1     Running   0           11s
```

Step 2 — Create ClusterIP Service

```
vi apache-service.yaml
```

A terminal window with a black background and green text. The first command is 'vi apache-service.yaml'. The second command is 'cat apache-service.yaml' which outputs the YAML configuration for a ClusterIP service named 'apache-service'. The configuration includes apiVersion: v1, kind: Service, metadata with name: apache-service, spec with type: ClusterIP, selector with app: apache, and ports with port: 80 and targetPort: 80. The prompt returns to 'ubuntu@master:~\$'.

```
ubuntu@master:~$ vi apache-service.yaml
[ubuntu@master:~$ cat apache-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: apache-service
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - port: 80
    targetPort: 80
ubuntu@master:~$
```

Apply:

```
kubectl apply -f apache-service.yaml
```

```
kubectl get svc
```

You should see:

```
apache-service    ClusterIP    10.xx.xx.xx    80/TCP
```

```
ubuntu@master:~$ kubectl apply -f apache-service.yaml
service/apache-service created
[ubuntu@master:~$ kubectl get svc
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
apache-service      ClusterIP     10.108.96.36  <none>         80/TCP     15s
kubernetes           ClusterIP     10.96.0.1     <none>         443/TCP    45h
ubuntu@master:~$
```

kubectl describe service apache-service

```
[ubuntu@master:~$ kubectl describe service apache-service
Name:                apache-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:            app=apache
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.108.96.36
IPs:                 10.108.96.36
Port:                <unset> 80/TCP
TargetPort:          80/TCP
Endpoints:           192.168.235.136:80
Session Affinity:    None
Events:              <none>
ubuntu@master:~$
```

—> kubectl get pods -o wide

```
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE   NOMINATED NODE   READINESS GATES
apache-pod    1/1     Running   0           6m2s  192.168.235.136  worker1  <none>           <none>
ubuntu@master:~$
```

Step 3 — Test

Create temporary pod:

```
kubectl run test --image=busybox -it --rm -- sh
Inside shell:
```

```
wget -qO- apache-service
```

If Apache HTML page appears → SUCCESS

```
ubuntu@master:~$ kubectl run test --image=busybox -it --rm -- sh
If you don't see a command prompt, try pressing enter.
/ # wget -qO- apache-service

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
/ #
/ #
```

ClusterIP exposes a pod internally within the cluster using a stable virtual IP.
Other pods can access the application using service name via Kubernetes DNS.

Task: 2 Expose an Nginx pod externally using a NodePort service.

This means:

Access website from your browser using **NodeIP:Port**

Step 1 — Create Nginx Pod YAML

```
vi nginx-pod.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi nginx-pod.yaml' and then 'cat nginx-pod.yaml'. The output shows a YAML configuration for a pod named 'nginx-pod' with a single container named 'nginx-container' using the 'nginx' image and exposing port 80. The prompt returns to 'ubuntu@master:~\$' after the command.

```
ubuntu@master:~$ vi nginx-pod.yaml
ubuntu@master:~$ cat nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80

ubuntu@master:~$
```

Apply:

```
kubectl apply -f nginx-pod.yaml
kubectl get pods
```

```
ubuntu@master:~$ kubectl apply -f nginx-pod.yaml
pod/nginx-pod created
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
apache-pod    1/1     Running   0           12m   192.168.235.136 worker1     <none>           <none>
nginx-pod     1/1     Running   0           8s    192.168.189.75  worker2     <none>           <none>
ubuntu@master:~$
```

Step 2 — Create NodePort Service YAML

vi nginx-nodeport.yaml

```
ubuntu@master:~$ vi nginx-nodeport.yaml
ubuntu@master:~$ cat nginx-nodeport.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
ubuntu@master:~$
```

nodePort range must be: 30000–32767

Apply:

```
kubectl apply -f nginx-nodeport.yaml
kubectl get svc
```

```
ubuntu@master:~$ kubectl apply -f nginx-nodeport.yaml
service/nginx-nodeport created
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES
apache-pod    1/1     Running   0           14m   192.168.235.136 worker1   <none>           <none>
nginx-pod     1/1     Running   0           2m9s  192.168.189.75  worker2   <none>           <none>
ubuntu@master:~$ kubectl get svc
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
apache-service ClusterIP    10.108.96.36  <none>        80/TCP           12m
kubernetes    ClusterIP    10.96.0.1     <none>        443/TCP          45h
nginx-nodeport NodePort     10.105.255.11 <none>        80:30007/TCP     21s
ubuntu@master:~$
```

Step 3 — Access Website

Get worker IP:

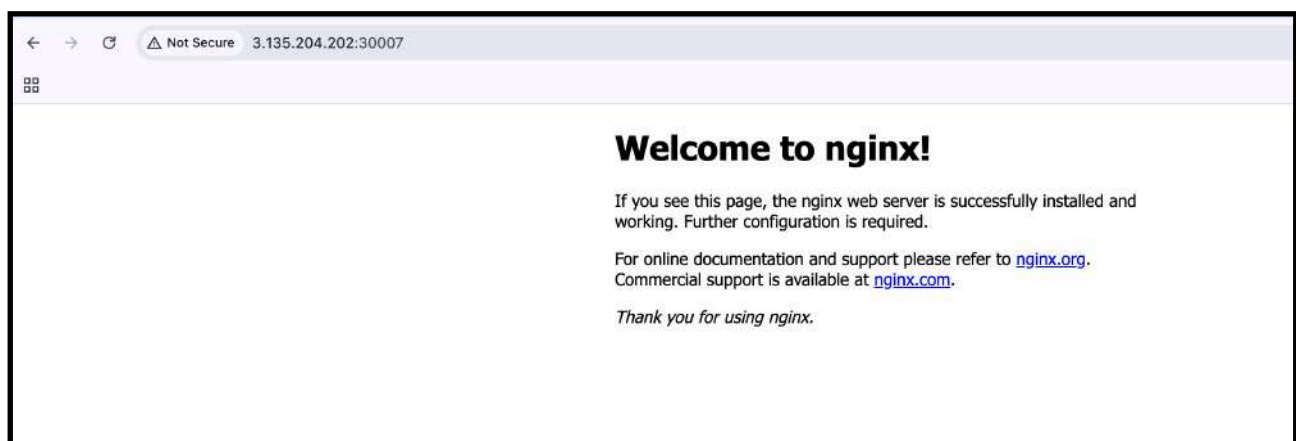
```
kubectl get nodes -o wide
```

Open browser:

`http://<WorkerIP>:30007` any worker ip inside cluster

You should see:

Welcome to nginx!



Task-3: Deploy a ReplicationController to maintain 3 replicas of an Nginx pod

Meaning: Kubernetes will always keep 3 nginx pods running automatically

Step 1 — Create ReplicationController YAML

```
vi nginx-rc.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user has entered 'cat nginx-rc.yaml'. The output is a YAML configuration for a ReplicationController. The configuration specifies 3 replicas, a selector for 'app: nginxrc', and a template with a container named 'nginx-container' using the 'nginx' image and port 80.

```
[ubuntu@master:~$ cat nginx-rc.yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginxrc
  template:
    metadata:
      labels:
        app: nginxrc
    spec:
      containers:
      - name: nginx-container
        image: nginx
        ports:
        - containerPort: 80

ubuntu@master:~$
```

Step 2 — Apply

```
kubectl apply -f nginx-rc.yaml
```

Check:

```
kubectl get pods
```

You should see **3 pods**:

```
nginx-rc-xxxxx  
nginx-rc-xxxxx  
nginx-rc-xxxxx
```

```
ubuntu@master:~$ kubectl apply -f nginx-rc.yaml  
replicationcontroller/nginx-rc created  
[ubuntu@master:~$ kubectl get pods  
NAME             READY   STATUS    RESTARTS   AGE  
apache-pod        1/1     Running   0           20m  
nginx-pod          1/1     Running   0           7m30s  
nginx-rc-2tzdr     1/1     Running   0           12s  
nginx-rc-hl865     1/1     Running   0           12s  
nginx-rc-klvqp     1/1     Running   0           12s  
ubuntu@master:~$
```

Step 3 — Prove RC works

Delete one pod manually:

```
kubectl delete pod <any-one-pod-name>
```

—> `kubectl delete pod nginx-rc-klvqp`

```

[ubuntu@master:~]$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
apache-pod    1/1     Running   0           20m
nginx-pod     1/1     Running   0           7m30s
nginx-rc-2tzdr 1/1     Running   0           12s
nginx-rc-hl865 1/1     Running   0           12s
nginx-rc-klvqp 1/1     Running   0           12s
[ubuntu@master:~]$ kubectl delete pod nginx-rc-klvqp
pod "nginx-rc-klvqp" deleted
[ubuntu@master:~]$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
apache-pod    1/1     Running   0           21m
nginx-pod     1/1     Running   0           9m19s
nginx-rc-2tzdr 1/1     Running   0           2m1s
nginx-rc-hl865 1/1     Running   0           2m1s
nginx-rc-nk45v 1/1     Running   0           4s
ubuntu@master:~$

```

Kubernetes will automatically create a new pod

This proves ReplicationController maintains desired replicas

—>ReplicationController ensures specified number of pod replicas are always running. If a pod fails or is deleted, it automatically recreates it.

Task-4: Scale the ReplicationController from 3 replicas → 5 replicas

I already created: from task 3

```
nginx-rc
```

```
replicas: 3
```

We will increase it to **5 running pods**

Method 1 — Command (fastest)

```
kubectl scale rc nginx-rc --replicas=5
```

Check:

```
kubectl get pods
```

Now you should see **5 pods**

```
nginx-rc-xxxxx
```

```
nginx-rc-xxxxx
```

```
nginx-rc-xxxxx
```

```
nginx-rc-xxxxx
```

```
nginx-rc-xxxxx
```

Method 2 — Edit YAML

—> we will do method 2

```
kubectl edit rc nginx-rc
```

Change:

replicas: 3
to

replicas: 5

Save → Kubernetes automatically creates 2 more pods

```
  app: nginxrc
  name: nginx-rc
  namespace: default
  resourceVersion: "23108"
  uid: fc075dee-855f-4d46-baf8-73588f1af3f6
spec:
  replicas: 5
  selector:
    app: nginxrc
  template:
    metadata:
      creationTimestamp: null
    labels:
      app: nginxrc
  spec:
```

Verify Scaling

```
kubectl get rc
```

```

nginx-rc-hl865v 1/1 Running 0
ubuntu@master:~$ kubectl edit rc nginx-rc
replicationcontroller/nginx-rc edited
ubuntu@master:~$ kubectl get rc
NAME          DESIRED   CURRENT   READY   AGE
nginx-rc      5         5         5       6m17s
[ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
apache-pod    1/1     Running   0          26m
nginx-pod     1/1     Running   0          13m
nginx-rc-2tzdr 1/1     Running   0          6m29s
nginx-rc-gk2mf 1/1     Running   0          31s
nginx-rc-hl865 1/1     Running   0          6m29s
nginx-rc-jg86d 1/1     Running   0          31s
nginx-rc-nk45v 1/1     Running   0          4m32s
ubuntu@master:~$

```

Scaling means increasing or decreasing the number of running pod replicas dynamically without downtime.

Task-5: Create a ReplicaSet to manage pods based on multiple labels (prod AND test)

This shows why **ReplicaSet > ReplicationController**
 Because ReplicaSet supports **advanced label selectors**

Step 1 — Create ReplicaSet YAML

```
vi rs-multilabel.yaml
```

```
nginx-rc-nk45v 1/1 Running 0 4m52s
ubuntu@master:~$ vi rs-multilabel.yaml
[ubuntu@master:~$ cat rs-multilabel.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-multi-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: prod
      type: test
  template:
    metadata:
      labels:
        env: prod
        type: test
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80

ubuntu@master:~$
```

Step 2 — Apply

```
kubectl apply -f rs-multilabel.yaml
```

Check:

```
kubectl get rs
kubectl get pods --show-labels
```

You should see pods like:

nginx-multi-rs-xxxxx	env=prod,type=test
nginx-multi-rs-xxxxx	env=prod,type=test
nginx-multi-rs-xxxxx	env=prod,type=test

```

ubuntu@master:~$ kubectl apply -f rs-multilabel.yaml
replicaset.apps/nginx-multi-rs created
[ubuntu@master:~$ kubectl get rs
NAME          DESIRED  CURRENT  READY  AGE
nginx-multi-rs 3         3        3      10s
[ubuntu@master:~$ kubectl get pods --show-labels
NAME          READY  STATUS   RESTARTS  AGE    LABELS
apache-pod    1/1    Running  0          31m    app=apache
nginx-multi-rs-7gcf5  1/1    Running  0          20s    env=prod,type=test
nginx-multi-rs-8k6ds  1/1    Running  0          20s    env=prod,type=test
nginx-multi-rs-smljw  1/1    Running  0          20s    env=prod,type=test
nginx-pod     1/1    Running  0          18m    app=nginx

```

Important Understanding

This ReplicaSet will only control pods having **BOTH** labels

env=prod AND type=test

If label mismatches → RS will ignore the pod

ReplicaSet uses label selectors (matchLabels/matchExpressions) to manage pods, unlike ReplicationController which supports only equality-based selectors.

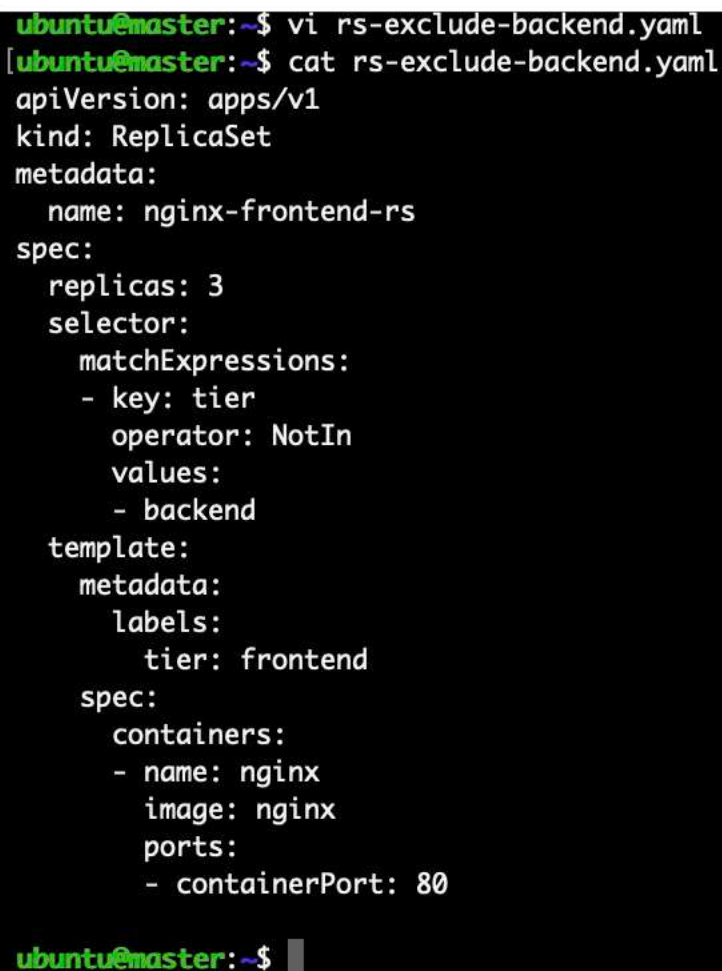
Task-6: Deploy a ReplicaSet that EXCLUDES pods with label backend

Here we use matchExpressions + NotIn operator

This is something ReplicationController cannot do — only ReplicaSet.

Step 1 — Create YAML

```
vi rs-exclude-backend.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi rs-exclude-backend.yaml' and then 'cat rs-exclude-backend.yaml'. The output shows a YAML configuration for a ReplicaSet named 'nginx-frontend-rs'. The configuration includes apiVersion: apps/v1, kind: ReplicaSet, metadata with name, spec with 3 replicas and a selector excluding the 'backend' tier, and a template with a container named 'nginx' using the 'nginx' image and port 80.

```
ubuntu@master:~$ vi rs-exclude-backend.yaml
ubuntu@master:~$ cat rs-exclude-backend.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-frontend-rs
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: tier
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80

ubuntu@master:~$
```

Step 2 — Apply

```
kubectl apply -f rs-exclude-backend.yaml
```

Check:

```
kubectl get rs
kubectl get pods --show-labels
```

```
ReplicaSet.apps/nginx-frontend-rs created
[ubuntu@master:~]$ kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
nginx-frontend-rs    3         3         3       12s
nginx-multi-rs       3         3         3       13m
[ubuntu@master:~]$ kubectl get pods --show-labels
```

```
ReplicaSet.apps/nginx-frontend-rs created
ubuntu@master:~$ kubectl get pods --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
nginx-frontend-rs-wv7zb             1/1     Running   0          5s    tier=frontend
nginx-frontend-rs-zdtcp             1/1     Running   0          5s    tier=frontend
nginx-frontend-rs-zpsxr             1/1     Running   0          5s    tier=frontend
nginx-multi-rs-bn4ld                1/1     Running   0          10s   env=prod,type=test
```

Now FINAL VALIDATION

—> Create backend pod

```
kubectl run backend-pod --image=nginx --labels="tier=backend"
kubectl get pods --show-labels
```

You will see backend pod appear.

```

nginx-rc-zvm89      1/1      Running    0          10s    app=nginxrc
[ubuntu@master:~$ kubectl run backend-pod --image=nginx --labels="tier=backend"
pod/backend-pod created
[ubuntu@master:~$ kubectl get pods --show-labels
NAME                                READY   STATUS    RESTARTS   AGE     LABELS
backend-pod                         1/1     Running   0          6s      tier=backend
nginx-frontend-rs-wv7zb            1/1     Running   0          2m5s    tier=frontend
nginx-frontend-rs-zdtcp            1/1     Running   0          2m5s    tier=frontend

```

—> Delete backend pod (should NOT recreate)

```

kubectl delete pod backend-pod
kubectl get pods

```

backend pod will NOT come back

```

nginx-rc-zvm89      1/1      Running    0          2m10s    app=nginxrc
[ubuntu@master:~$ kubectl delete pod backend-pod
pod "backend-pod" deleted
[ubuntu@master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-frontend-rs-wv7zb            1/1     Running   0          2m27s
nginx-frontend-rs-zdtcp            1/1     Running   0          2m27s
nginx-frontend-rs-zpsxr            1/1     Running   0          2m27s
nginx-multi-rs-bn4ld               1/1     Running   0          2m32s

```

—> Delete frontend pod (should recreate)

```

kubectl delete pod nginx-frontend-rs-wv7zb
kubectl get pods -l tier=frontend

```

—> New frontend pod will appear automatically

```

nginx-rc-zvm89      1/1      Running    0          2m32s
[ubuntu@master:~$ kubectl delete pod nginx-frontend-rs-wv7zb
pod "nginx-frontend-rs-wv7zb" deleted
[ubuntu@master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-frontend-rs-d8tf8  1/1     Running   0          5s
nginx-frontend-rs-zdtcp  1/1     Running   0         4m10s
nginx-frontend-rs-zpsxr  1/1     Running   0         4m10s

```

```

nginx-rc-zvm89      1/1      Running    0          4m15s
[ubuntu@master:~$ kubectl get pods --show-labels
NAME                READY   STATUS    RESTARTS   AGE   LABELS
nginx-frontend-rs-d8tf8  1/1     Running   0         23s   tier=frontend
nginx-frontend-rs-zdtcp  1/1     Running   0        4m28s   tier=frontend
nginx-frontend-rs-zpsxr  1/1     Running   0        4m28s   tier=frontend

```

ReplicaSet excludes backend pods using NotIn operator.

Backend pod was not recreated after deletion, proving it is not managed.

Frontend pod was recreated, proving it is managed by ReplicaSet.

Task 7 : Test load balancing across multiple pods using NodePort

STEP 1 — Create ReplicaSet file

Open editor:

```
vi lb-rs.yaml
```

```

ubuntu@master:~$ vi lb-rs.yaml
[ubuntu@master:~$ cat lb-rs.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: lb-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lbn
  template:
    metadata:
      labels:
        app: lbn
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80

ubuntu@master:~$ █

```

Apply:

```

kubectl apply -f lb-rs.yaml
kubectl get pods -l app=lbn -o wide

```

```

ubuntu@master:~$ kubectl apply -f lb-rs.yaml
replicaset.apps/lb-rs created
ubuntu@master:~$ kubectl get pods -l app=lbn -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
lb-rs-2fphw	1/1	Running	0	7s	192.168.189.88	worker2	<none>	<none>
lb-rs-mm6r6	1/1	Running	0	7s	192.168.235.149	worker1	<none>	<none>
lb-rs-s8hk9	1/1	Running	0	7s	192.168.189.89	worker2	<none>	<none>

```

lb-rs-s8hk9 1/1 Running 0 7s 192.168.189.89 worker2 <none> <
[ubuntu@master:~]$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP             NODE          NOMINATED NODE   R
lb-rs-2fphw   1/1     Running   0           48s   192.168.189.88 worker2        <none>            <
lb-rs-mm6r6   1/1     Running   0           48s   192.168.235.149 worker1        <none>            <
lb-rs-s8hk9   1/1     Running   0           48s   192.168.189.89 worker2        <none>            <
[ubuntu@master:~]$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
lb-rs-2fphw   1/1     Running   0           58s
lb-rs-mm6r6   1/1     Running   0           58s
lb-rs-s8hk9   1/1     Running   0           58s
ubuntu@master:~$

```

Wait until you see **3 running pods**

STEP 2 — Create NodePort Service (to access pods)

Open a new file:

```
vi lb-service.yaml
```

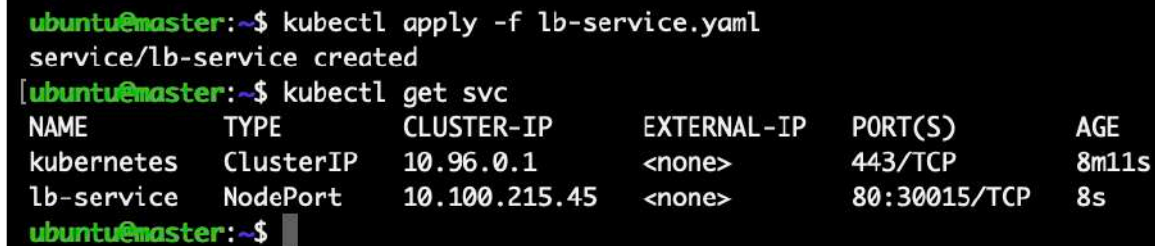
```

lb-rs-s8hk9 1/1 Running 0 58s
ubuntu@master:~$ vi lb-service.yaml
[ubuntu@master:~]$ cat lb-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: lb-service
spec:
  type: NodePort
  selector:
    app: lb
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30015
ubuntu@master:~$

```

Now apply it:

```
kubectl apply -f lb-service.yaml
kubectl get svc
```

A terminal window with a black background and green text. It shows the execution of two kubectl commands. The first command, 'kubectl apply -f lb-service.yaml', results in the output 'service/lb-service created'. The second command, 'kubectl get svc', displays a table of services. The table has six columns: NAME, TYPE, CLUSTER-IP, EXTERNAL-IP, PORT(S), and AGE. It lists two services: 'kubernetes' (ClusterIP 10.96.0.1, port 443/TCP, age 8m11s) and 'lb-service' (NodePort 10.100.215.45, port 80:30015/TCP, age 8s).

```
ubuntu@master:~$ kubectl apply -f lb-service.yaml
service/lb-service created
[ubuntu@master:~$ kubectl get svc
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes          ClusterIP     10.96.0.1     <none>         443/TCP          8m11s
lb-service          NodePort      10.100.215.45 <none>         80:30015/TCP     8s
ubuntu@master:~$
```

STEP 3 — Get Node IP

Run:

```
kubectl get nodes -o wide
```

Test the service (open website)

Use the IP you saw and the NodePort **30015**

Run this command multiple times:

```
curl http://<NodeIP>:30015
```

```

lb-service NodePort 10.100.215.45 <none> 80:30015/TCP 8s
ubuntu@master:~$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP
master Ready control-plane 46h v1.30.14 172.31.14.17 <none>
worker1 Ready <none> 46h v1.30.14 172.31.9.88 <none>
worker2 Ready <none> 46h v1.30.14 172.31.9.239 <none>
ubuntu@master:~$ curl 172.31.9.88:30015
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@master:~$

```

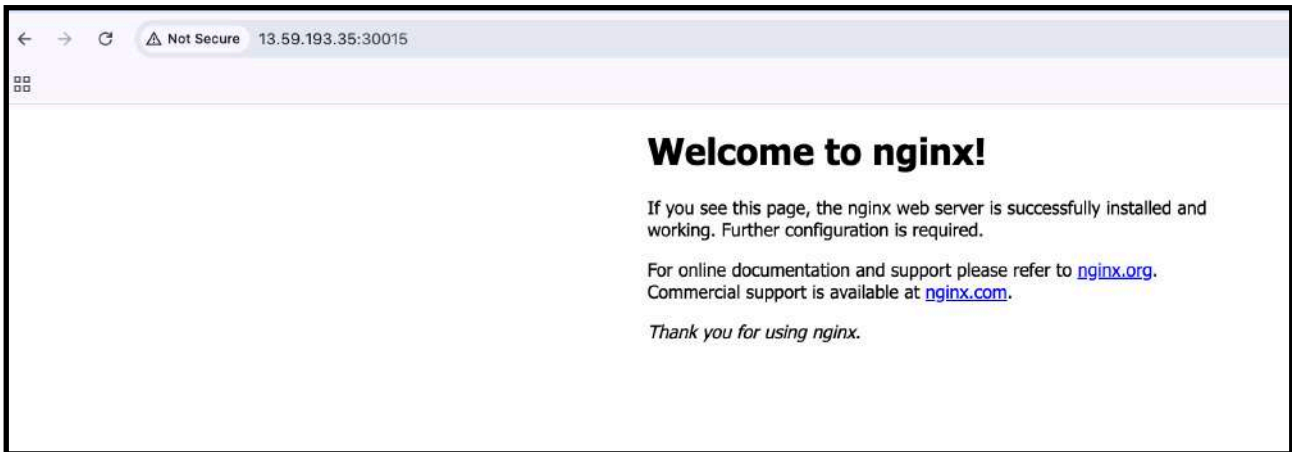
—> kubectl describe service lb-service

```

Error from server (NotFound): services "lb-service.yaml" not found
ubuntu@master:~$ kubectl describe service lb-service
Name: lb-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=lb
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.100.215.45
IPs: 10.100.215.45
Port: <unset> 80/TCP
TargetPort: 80/TCP
NodePort: <unset> 30015/TCP
Endpoints: 192.168.189.88:80,192.168.189.89:80,192.168.235.149:80
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
ubuntu@master:~$

```

—> check from your any worker public ip:30015



Task 8.Delete a Replication Controller without affecting the running pods.

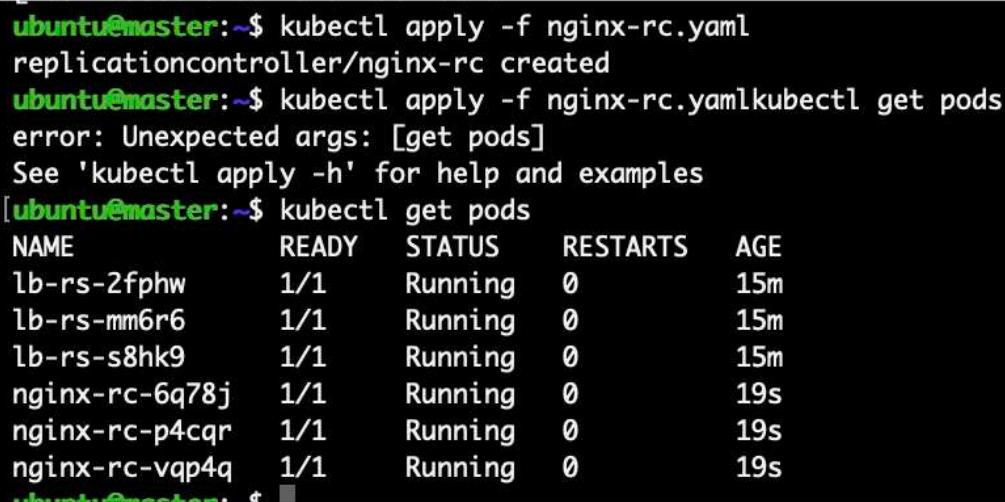
—> I already have a rc yaml file

```
apache-service.yaml  createpod.yaml  envpod.yaml  firstpod.yaml
[ubuntu@master:~]$ cat nginx-rc.yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginxrc
  template:
    metadata:
      labels:
        app: nginxrc
    spec:
      containers:
      - name: nginx-container
        image: nginx
        ports:
        - containerPort: 80

ubuntu@master:~$
```

STEP 1 — Create the ReplicationController

```
kubectl apply -f nginx-rc.yaml
kubectl get pods
```



A terminal window showing the execution of kubectl commands. The first command creates a ReplicationController named 'nginx-rc'. The second command attempts to get pods but has a syntax error. The third command correctly lists the pods, showing six nginx-rc pods in a 'Running' state.

```
ubuntu@master:~$ kubectl apply -f nginx-rc.yaml
replicationcontroller/nginx-rc created
ubuntu@master:~$ kubectl apply -f nginx-rc.yamlkubect get pods
error: Unexpected args: [get pods]
See 'kubectl apply -h' for help and examples
[ubuntu@master:~$ kubectl get pods
NAME                READY    STATUS    RESTARTS   AGE
lb-rs-2fphw         1/1      Running   0           15m
lb-rs-mm6r6         1/1      Running   0           15m
lb-rs-s8hk9         1/1      Running   0           15m
nginx-rc-6q78j      1/1      Running   0           19s
nginx-rc-p4cqr      1/1      Running   0           19s
nginx-rc-vqp4q      1/1      Running   0           19s
ubuntu@master:~$
```

Wait until you see **3 pods running** like:

```
nginx-rc-xxxxx    Running
nginx-rc-xxxxx    Running
nginx-rc-xxxxx    Running
```

STEP 2 — Delete RC but KEEP the pods

Now run:

```
kubectl delete rc nginx-rc --cascade=orphan
```

This means:

- RC removed
- Pods remain running

```
nginx-rc-vqp4q 1/1 Running 0 19s
[ubuntu@master:~$ kubectl get rc
NAME          DESIRED  CURRENT  READY  AGE
nginx-rc      3        3        3      2m40s
[ubuntu@master:~$ kubectl delete rc nginx-rc --cascade=orphan
replicationcontroller "nginx-rc" deleted
[ubuntu@master:~$ kubectl get pods
NAME          READY  STATUS   RESTARTS  AGE
lb-rs-2fphw   1/1    Running  0         18m
lb-rs-mm6r6   1/1    Running  0         18m
lb-rs-s8hk9   1/1    Running  0         18m
nginx-rc-6q78j 1/1    Running  0         3m
nginx-rc-p4cqr 1/1    Running  0         3m
nginx-rc-vqp4q 1/1    Running  0         3m
[ubuntu@master:~$ kubectl get rc
No resources found in default namespace.
ubuntu@master:~$
```

STEP 3 — Verify

Check RC:

```
kubectl get rc
```

Check pods:

```
kubectl get pods
```

You should see:

No replicationcontrollers found
but pods still exist:

```
nginx-rc-xxxxx Running
```

```
nginx-rc-xxxxx    Running
nginx-rc-xxxxx    Running
```

```
[ubuntu@master:~$ kubectl get rc
No resources found in default namespace.
[ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
lb-rs-2fphw   1/1     Running   0           19m
lb-rs-mm6r6   1/1     Running   0           19m
lb-rs-s8hk9   1/1     Running   0           19m
nginx-rc-6q78j 1/1     Running   0           4m5s
nginx-rc-p4cqr 1/1     Running   0           4m5s
nginx-rc-vqp4q 1/1     Running   0           4m5s
ubuntu@master:~$
```

The `--cascade=orphan` option deletes the ReplicationController but preserves its pods by removing ownership metadata, making the pods standalone resources.